Bounty of Juan — Presentation Outline (10 min)

1. Introduction (1 min)

- What is the game? 2D top-down survivor-style shooter
- Player is Juan — survive waves of enemies as long as possible
- Built in C++17 using the SFML graphics library
- Team of 4: John, Brayden, Tristan, Nathan

---

2. Live Demo (2 min)

- Launch from welcome screen → click Play
- Show movement, auto-shooting, enemies chasing and attacking
- Take some damage to show health bar
- Press ESC to show pause overlay
- Die → show results screen with stats
- Click Main Menu → back to welcome, ready to play again

---

3. Architecture Overview (1.5 min)

- **State machine** drives the whole game: `welcome → game → paused / results`
- `Game` class owns the state and delegates `handleInput`, `update`, `render` each frame (game.cpp, lines 15-90)
- `main.cpp` runs the loop (lines 13-80), handles the infinite tiling background, and keeps the camera centered on Juan
- Assets (textures, fonts) loaded once in `main.cpp` and passed down by reference

```
main.cpp
  └── Game (state machine)
        ├── Welcome  (title screen)
        ├── Play     (gameplay — Juan + enemies)
        └── ResultsScreen  (game over screen)
```

Sprint Refactoring: Both Juan and Enemy now inherit from a `Character` base class (character.cpp) that handles shared health mechanics (damage, death detection). This eliminates code duplication and enables polymorphic design patterns for future features like unified stat tracking or character collections.

4. Contributor Sections (4.5 min — ~1 min each)

Nathan — Project Skeleton & Core Engine

- Created the very first commit: project folder structure, Makefile, and all empty source/header files
- Built the core game loop in `main.cpp` (lines 13-80) including the SFML window, delta-time clock, and frame cycle
- Implemented the infinite tiling background system that seamlessly renders background tiles across world space
- Added the camera system in `main.cpp` that keeps the view centered on Juan as he moves
- Built out Juan's WASD movement with normalized diagonal movement so speed is consistent at all angles (juan.cpp, lines 54-76)
- Constructed the `Game` state machine connecting `welcome`, `game`, `paused`, and `results` states (game.cpp, lines 40-90)
- Implemented the Welcome screen with the western-font title and wired it into the state machine

Brayden — Enemy System & Combat

- Built the `Enemy` class from scratch (enemy.cpp, lines 40-61): texture loading, sprite scaling, and two constructors (one initializer, one for all subsequent enemies)
- Enemies use a shared static pointer to Juan and the window so all instances stay in sync without redundant references
- Enemy movement uses vector normalization (the same math as Juan's movement) to walk smoothly toward the player
- Enemies rotate to face Juan at all times using arc-sine angle calculation
- Implemented the full `Projectile` class (projectile.cpp, lines 17-30) and wired Juan to auto-shoot toward the nearest enemy every 15 frames
- Added collision detection between Juan's projectiles and enemy hitboxes (play.cpp, lines 75-100)
- Enemies have 3 HP and are removed from the heap when killed, with memory managed via pointer vector cleanup
- Integrated all enemies into `Play` with `initializeEnemyList`, `addEnemy`, `updateAllEnemies`, and `destroyEnemyList`
- **Sprint Update**: Refactored `Enemy` to inherit from `Character` base class (character.cpp), eliminating code duplication and enabling polymorphic character handling

Tristan — Button Class, Stats & HUD

- Added initial Juan stats structure to `juan.h` and `juan.cpp` (first stat tracking groundwork)
- Added Doxygen-style file headers and comments across all header files for documentation
- Implemented the full `Button` class (button.cpp, lines 18-26) with three visual states (normal, hovered, clicked), mouse hit detection, and color feedback

- Built the `Results` singleton stat tracker (results.cpp, lines 14-30) using a static instance pattern — tracks enemies spawned/killed, shots fired, shots missed, and time survived across the entire session
- Wired stat tracking calls into `juan.cpp` (shot fired/missed) and `play.cpp` (enemy spawned/killed, time update)
- Added the in-game HUD overlay that displays live time, enemies killed, shots fired, and accuracy in the top-left corner during gameplay

John — Screens, Health System & Polish

- Built the Welcome screen with the styled western-font title and Play button
- Built the Results screen showing final stats pulled from the tracker
- Added Juan's health system (100 HP), health bar, and the death → results transition (juan.cpp, lines 21-110)
- Enemy contact damage with windup animation (enemy.cpp, lines 140-180) (scales to 110% before striking)
- ESC pause with a dimmed overlay and resume hint (game.cpp, lines 96-115)
- Restart resets health, clears enemies, and wipes stats for a clean new game
- Resolved merge conflicts across all branches throughout the project

---

5. Code Highlight (30 sec)

Pick one interesting snippet to show — suggested: the state machine switch in `game.cpp` or the enemy contact damage + scale animation in `enemy.cpp`

---

6. Challenges & What We Learned (30 sec)

- Coordinating across 4 branches with overlapping files (`game.cpp` , `play.cpp`) required careful merging
- SFML's coordinate system is world-space, so enemy spawns had to account for Juan's position rather than raw screen dimensions
- Separating game logic (update) from rendering was important — shooting was accidentally tied to render, causing it to fire even while paused

---

7. Q&A (30 sec buffer)